

SEN381 ASSIGNMENT 2

GROUP E

CHRIS FOURIE

LISA VERNON

PANDORA GREYLING

Table of Contents

1. Task 1: Researching Design Quality and Design Patterns	2
1.1 Architectural Foundations of Internal Design Quality.....	2
1.2 Comparative Evaluation of Candidate Design Patterns.....	3
1.3 Evidence-Based Recommendations Feeding Milestone 2	4
2. Task 2: Researching Persistence and Data-Integrity Decisions	5
2.1 Selected Core Business Operation & Failure Modes	5
2.2 Correctness Mechanisms & Engineering Trade-Offs	6
2.3 Implementation Approaches & M2 Recommendations	10
3. Task 3: Researching APIs and Integration Decisions	12
3.1 Framing the CivicConnect Integration Problem	12
3.2 Comparative Evaluation of Integration Mechanisms.....	13
3.3 Evidence-Based Integration Recommendation for M2	16
4. Task 4: Researching Collaborative Engineering and CI Controls	18
4.1 Software Configuration Management & Collaborative Integration	18
4.2 Continuous Integration Pipeline & Automated Quality Gates	20
4.3 Recommended CivicConnect Team Workflow for M2/M3	23
5. Task 5: The Research-to-Decision Map	26
5.1 The Master Traceability & Decision Matrix.....	26
6. Responsible AI Usage Register & Academic Integrity	27
6.1 Responsible AI Usage Register.....	27
6.1.1 Student Attestation & Critical Oversight Case Critique	29
6.2 Formal Academic & Standards References.....	29

1. Task 1: Researching Design Quality and Design Patterns

1.1 Architectural Foundations of Internal Design Quality

1.1.1 Coupling and Cohesion

Coupling is the extent to which software parts or modules depend on each other, while cohesion measures how closely the internal components inside a single module work together to achieve a job. The interaction between different components and classes shows the complexity of the design. There may be multiple designs, but the complexity level of the design may decide the better solution. Coupling and cohesion which shows the connection of classes and the capability of classes, control the complexity of the design (Saxena and Kumar, 2012). A good software object-oriented design will be based on low coupling and high cohesion (less inter-dependency with high functionality)

Cohesion and coupling have a significant influence on the difficult of changing requirements or updates to the project. With a tightly coupled system, a system with many dependent classes, and low cohesion, low strength to complete a task alone, one update will probably result in the need to update many different classes. The likelihood of making a mistake when updating or adding new features increases with a tightly coupled systems.

1.1.2 SOLID Principles

Single Responsibility Principle (SRP) states that a class should be designed to fulfil a single function or purpose. This ensures that each class remains dedicated to a single task, which simplifies its maintenance and potential for expansion. By enabling SRP modifications are less likely to affect other parts of the system which reduces the risk of introducing new errors (Sarcar, 2021). CivicConnect's requester capabilities introduce two requirements which could fall under the responsibility of one class, submitting a new service request and handling feedback, capturing/validating a new request and communicating status changes back to the requester. This class would have two reasons to change, a new required field or a new notification channel.

Open/Closed Principle (OCP) states that software components should be designed to allow future extension without requiring to change pre-existing code. Improvements to a module should be done through new code rather than changing existing code. OCP is important for ensuring the system remains stable while expanding it. CivicConnect uses category sets and lists for service request capturing, sorting and filtering. Hard coding the category logic will require an update to every instance of existing filtering or selection code when a new category is added. A new category should not require rework of existing code.

Dependency Inversion Principle (DIP) states that software components should not rely on other components but rather depend on abstractions.

1.1.3 CivicConnect Design Problems

Design Problem 1: Status-transition logic risks being entangled with unrelated responsibilities

CivicConnect requires staff to assign or accept responsibility for a request, update request status through controlled transitions, record relevant actions, comments or resolution information and resolve or close requests where authorized. While these actions take place, requesters must also be able to receive meaningful feedback when their request status is changed.

Changing the status of a request will trigger several different operations, checking whether the transition follows a controlled pattern (eg. Changes from “New” to “Assigned”, not directly to “Closed”), recording the action for audit purposes and ensuring that the requester is notified of the changes. If all this logic presides in a single class or method, it would violate SRP. This would increase the risk of the intended change affecting other components as well as making unit testing more difficult.

Design Problem 2: Category-specific behaviour is likely to be implemented as scattered conditional logic

CivicConnect requires requesters to categorise a request using a controlled category mechanism and lists example categories including facility faults, damaged equipment, security concerns, IT support, maintenance issues, lost property and other operational requests which leaves the category set open. Management will need to be able to view the category by the list or other justified dimensions.

Different categories may require different behaviors. A security concern may require different required fields and level of urgency as opposed to a lost property concern. Different categories may also need to route to different staff groups. This may likely lead to conditional branching (if/switch statements) across request creation and staff assignment. Adding a new category would require locating and editing each conditional logic instance across multiple classes. This violates OCP and introduces regression risk by editing already reviewed and tested code.

1.2 Comparative Evaluation of Candidate Design Patterns

Candidate A: Observer Pattern

The Observer Pattern defines interactions between two objects, observers and subjects. The observers react to changes in the state of the subjects. The observer assigns to the subject and is notified when the subject's state changes

To prevent coupling tight between objects where the observer is applied, an event that describes the change can be passed instead of passing the identity of the changed subject. The coupling between subjects and observers is abstract and minimal (Naumovich, 2002). The observer pattern would also show an improvement to cohesion by separating the detection of change from the reaction to change. The subject stays focused on managing its own state and knowing when a change has occurred. It no longer contains the logic for reactions. The observers, respectively, focus on their specific reaction. This pattern would improve the maintainability as changes to an observer's reactions only need to be implemented for that specific observer. The subject and other observers will remain unaffected (Gamma et al., 1994). The extensibility will also show improvement as a new observer class can be created without modifying existing code. Each observer can be tested in separately from the other observers. The subject can also be tested using fake observers to ensure it notifies observers when a change occurs.

Observers have no knowledge of each other's presence and cannot see the risk of changing the subject. An operation on the subject may cause a cascade of updates to observers and their dependent objects. Dependency criteria that aren't well defined or maintained can lead to invalid updates, which can be difficult to find (Gamma et al., 1994). A misuse risk of the Observer pattern is using it solely, because changing the status sounds like an event, without checking whether the reactions are independent or a mandatory step.

If CivicConnect only has a small, fixed reaction set to a status change and it is unlikely to grow, a simpler more direct approach may satisfy all the requirements with less indirection to maintain.

1.3 Evidence-Based Recommendations Feeding Milestone 2

Moving into Milestone 2 an Observer design pattern would fit our use case the best.

2. Task 2: Researching Persistence and Data-Integrity Decisions

2.1 Selected Core Business Operation & Failure Modes

2.1.1 Selected Operation: Atomic Service Request Status Transition & Technician Assignment with Audit Ledgering

To investigate persistence correctness and data integrity in CivicConnect, the core business operation selected is the atomic execution of a service request status transition coupled with technician assignment and mandatory audit trail logging. This operation directly operationalises Functional Requirements:

- **FR-009:** Assign or accept responsibility for a request.
- **FR-010:** Update request status through controlled transitions.
- **FR-011:** Record relevant actions, comments, or resolution details.
- **NFR-006:** Tamper-evident auditability and compliance.

2.1.2 Concrete Failure Modes and Downstream Operational Consequences

In a municipal service platform, this operation mutates multiple distinct data structures simultaneously: updating the operational ticket record, modifying technician queue counters, and appending an immutable entry to an audit table. Executing this operation without rigorous persistence controls creates three severe failure modes:

1. **Partial Write Failure (The Broken Ledger Anomaly):**

Failure Mechanism: The application updates the operational record in ServiceRequests (changing status = 'ASSIGNED' and assigned_technician_id = 'TECH-402'), but the subsequent insertion into RequestAuditLog fails due to an unhandled constraint violation, database timeout, or process termination.

Consequence: The ticket is assigned, but there is no historical record of who assigned it, when it was assigned, or why. This directly violates South African statutory compliance under the **Protection of Personal Information Act (POPIA Act 4 of 2013)**, which mandates non-repudiation and accountability when processing citizen service records.

2. **Technician Workload Deadlock (Ghost Assignments):**

Failure Mechanism: The system increments the assigned technician's active workload counter in their profile table, but the operational ticket update rolls back or fails validation.

Consequence: The technician is falsely marked at maximum capacity in dispatch

dashboards, preventing them from receiving urgent community repair tasks, while the ticket itself remains stranded in an unassigned state.

3. **State Machine Bypass and Illegal State Mutation:**

Failure Mechanism: If state transitions rely solely on superficial application-level flags without database-enforced integrity checks, concurrent operations or malformed API calls can force a ticket directly from SUBMITTED to RESOLVED, skipping mandatory technician triage and site inspection.

Consequence: Requests are marked "resolved" without work being performed, destroying citizen trust and invalidating management SLA performance metrics (FR-013).

2.2 Correctness Mechanisms & Engineering Trade-Offs

Under relational database theory (Kleppmann, 2017; Date, 2004; Bernstein et al., 1987), preserving correctness during multi-entity mutations requires encapsulating the entire operation within an explicit ACID transaction boundary:

- **Atomicity:** The operational mutation of ServiceRequests and the append-only write to RequestAuditLog are bound into an indivisible unit of work (BEGIN TRANSACTION ... COMMIT). If either operation fails, the database engine automatically executes ROLLBACK, guaranteeing that partial or corrupted states cannot persist.
- **Consistency:** The transaction enforces all relational invariants: referential foreign key integrity (verifying the assigned technician exists and possesses an active account) and table CHECK constraints enforcing the allowed FSM state enum values.
- **Isolation:** The transaction guarantees that concurrent dispatchers cannot read intermediate, uncommitted status changes. Group E evaluated three SQL transaction isolation levels:
 - *Read Uncommitted:* Vulnerable to "dirty reads" (reading uncommitted state transitions that are subsequently aborted). Rejected as dangerous for municipal audit records.
 - *Read Committed (Recommended):* Guarantees transactions only read committed data, preventing dirty reads while avoiding the heavy locking overhead of higher isolation levels. When combined with Optimistic

Concurrency Control, this provides the optimal balance of data correctness and low query latency (≤ 500 ms per NFR-001).

- **Serializable:** Complete serial execution. While providing maximum theoretical isolation, it introduces severe lock contention, frequent transaction aborts, and catastrophic latency degradation under concurrent multi-user load.
- **Durability:** Once committed, transaction records are written to non-volatile storage via the database engine's Write-Ahead Log (WAL), ensuring that state changes survive physical server power losses or container restarts.

2.2.2 Multi-Layer Validation & Defence-in-Depth

Enforcing validation at only a single layer is a recognized architectural vulnerability. CivicConnect implements a defence-in-depth validation strategy across three distinct tiers:

Architectural Tier	Validation Scope & Mechanisms	Strengths	Consequences of Relying Solely on this Tier
Tier 1: Client / UI Layer	Form-level syntax validation: required fields, string length caps, regex matching on contact numbers, allowed attachment file types (.jpg, .png, .pdf).	Instantaneous user feedback without server round-trip latency; optimises user experience.	Critical Security Flaw: Client validation is trivially bypassed via curl, Postman, or automated script injection. Relying solely on client validation exposes the platform to database corruption and code injection.
Tier 2: Application / Service Layer	Domain invariants & business logic: verifying current ticket status against FSM transition guards (e.g., asserting state is TRIAGED before permitting transition to ASSIGNED), role-based authorization verification (RBAC), and technician workload limits.	Context-aware; can query business rules and external services; generates clear, localized, user-friendly domain error messages.	Vulnerable to Direct Access: Any direct database patch, manual SQL data fix, or background batch migration bypasses application logic, risking invalid state generation.
Tier 3: Database	Declarative integrity constraints: NOT	Inviolable safety net;	Poor User Experience: Database

Engine Layer	NULL columns, FOREIGN KEY ... ON DELETE RESTRICT, CHECK (status IN ('SUBMITTED', 'TRIAGED', 'ASSIGNED', 'IN_PROGRESS', 'RESOLVED', 'CLOSED')), and immutable trigger constraints.	enforced globally regardless of entry path (API, migration, admin CLI); ACID-protected.	errors are cryptic, slow to return across the network, impossible to map directly to UI input fields, and can leak underlying table structures to end users.
---------------------	---	---	--

2.2.3 Consistency, Concurrency, and Race Conditions (OCC vs. Pessimistic Locking)

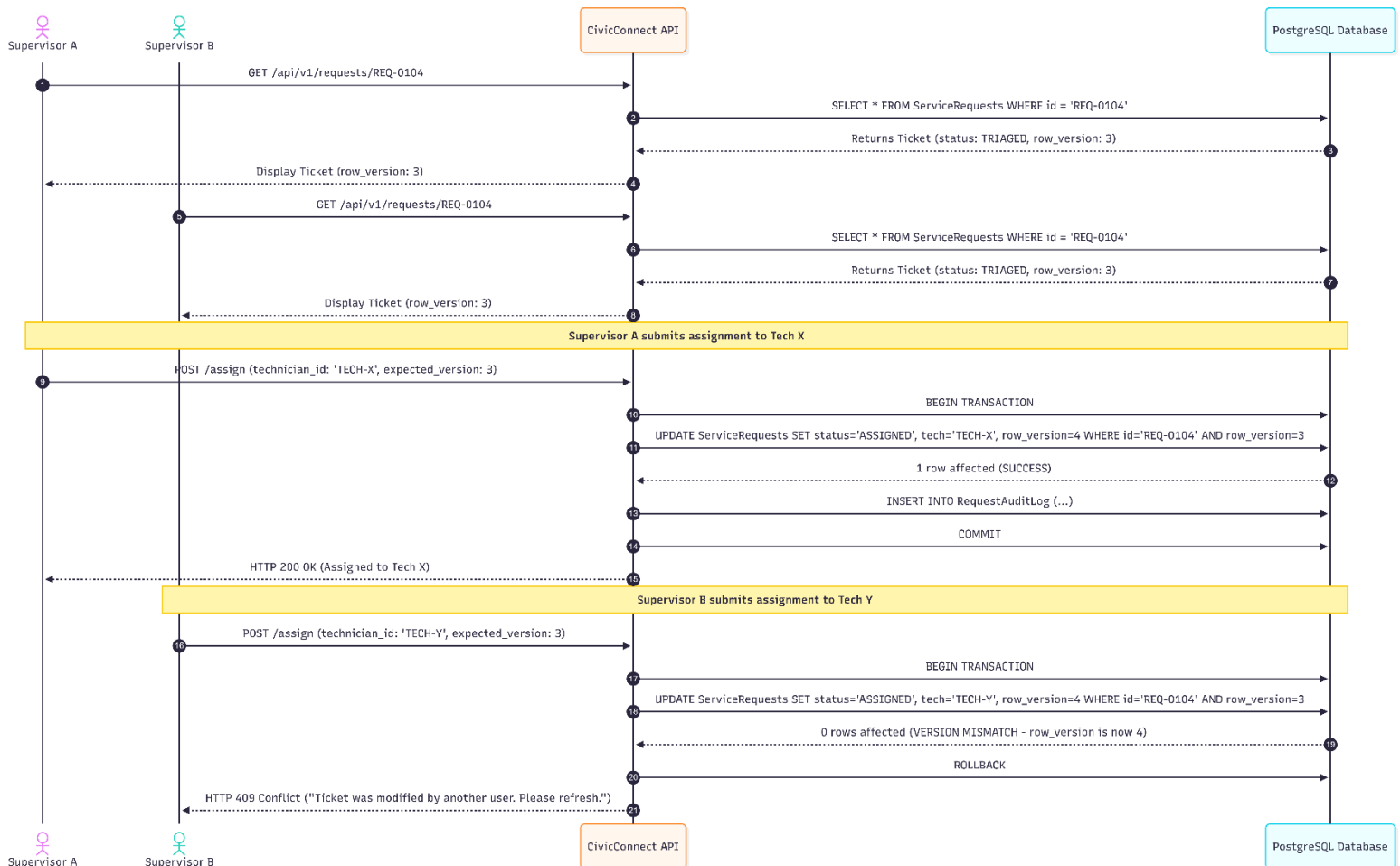
In a multi-user dispatch centre, concurrent operations create the classic "Lost Update" anomaly: Supervisor A and Supervisor B simultaneously view ticket REQ-0104 (status TRIAGED). Supervisor A assigns the ticket to Technician X at 10:00:01. Supervisor B assigns the ticket to Technician Y at 10:00:02. Without concurrency control, Supervisor B silently overwrites Supervisor A's assignment, leaving Technician X with an orphaned dispatch.

Group E evaluated two primary concurrency mechanisms:

1. **Pessimistic Locking (SELECT FOR UPDATE):** The first supervisor's transaction places an exclusive row-level lock on the ticket, forcing all other transactions to block until release.
Trade-off: While it guarantees serialised updates, pessimistic locking holds open active database connections during user think-time or slow network transfers. In a cloud-hosted environment with limited database connection pools (such as free-tier PostgreSQL instances capped at 20 concurrent connections), this leads to connection pool exhaustion, request timeouts, and cascading deadlocks.
2. **Optimistic Concurrency Control (OCC - Recommended):** Relies on a dedicated row_version integer column (or database commit identifier xmin). Transactions read data without taking locks. When an update is executed, the query includes the version token in its predicate:

```
UPDATE ServiceRequests
SET status = 'ASSIGNED',
    assigned_technician_id = 'TECH-402',
    row_version = row_version + 1,
    updated_at = NOW()
WHERE id = 'REQ-0104' AND row_version = 3;
```

If another supervisor modified the row in the interim, row_version has already incremented to 4. The SQL engine matches 0 rows. The application detects that zero rows were affected, immediately aborts the transaction, and returns an HTTP 409 Conflict status code, prompting the user to refresh their view with the latest state.



2.2.4 Caching Correctness vs. Staleness Trade-Off

Caching is frequently introduced as an unexamined performance reflex. In CivicConnect, caching must be evaluated against the strict trade-off between read latency and data staleness:

- **Active Ticket Operational State (STRICTLY DO NOT CACHE):** Ticket status, technician assignment, and resolution fields are highly mutable and drive operational dispatching. Caching these records in an in-memory cache (e.g. Redis or client memory) introduces a dangerous "stale read" window. If a supervisor acts upon a cached status showing a ticket as "Unassigned" when it was assigned

seconds prior, duplicate dispatches occur. Real-time operational data must always query the primary relational database directly.

- **Static Reference Data (JUSTIFIED CACHING):** Service categories (e.g. "Pothole", "Water Leak"), department definitions, municipal office locations, and static SLA target tables change infrequently (weeks or months). Caching these datasets in application memory with a Time-To-Live (TTL) of 60 minutes and an administrative cache-purge hook drastically reduces database load without introducing data integrity risks.

2.3 Implementation Approaches & M2 Recommendations

2.3.1 Comparative Evaluation of Persistence Implementation Approaches

Evaluation Criterion	Approach 1: Active Record with Implicit ORM Flushes	Approach 2: Repository Pattern with Explicit Unit-of-Work (Recommended)
Architectural Separation	Domain entities inherit directly from ORM base classes (<code>request.Save()</code>). Business logic is tightly coupled to SQL schema details.	Domain entities are pure domain objects (POCOs/plain classes). Persistence logic is encapsulated behind <code>IServiceRequestRepository</code> abstractions.
Transaction Boundary Control	Implicit and opaque. ORMs flush queries automatically based on internal change trackers, risking partial writes if an unhandled error occurs mid-routine.	Explicit and deterministic. Transaction boundaries (<code>BeginTransaction</code> , <code>Commit</code> , <code>Rollback</code>) are directly declared and auditable in code.
Unit Testability	Poor. Testing domain rules requires spinning up live database instances or heavy in-memory database shims.	Superb. Domain logic and state machine guards are tested in milliseconds using in-memory mock repositories.
Complexity & Overhead	Low initial setup overhead; rapid for trivial CRUD prototypes.	Requires interface declarations and explicit Unit-of-Work orchestration classes; slightly higher initial boilerplate.
Downstream Risk	Leaks database concerns into domain services; high risk of N+1 query problems and uncontrolled schema coupling in M3.	Decouples business rules from the underlying persistence provider, enabling zero-risk database migrations or schema adjustments.

2.3.2 Evidence-Based Persistence Recommendation Feeding Milestone 2

1. **Persistence Pattern:** Adopt the **Repository Pattern with an Explicit Unit-of-Work Transaction Boundary** to guarantee atomic multi-table writes across ServiceRequests and RequestAuditLog.
2. **Concurrency Model:** Implement **Optimistic Concurrency Control (OCC)** using a dedicated row_version integer column, returning HTTP 409 Conflict on concurrent collisions.
3. **Integrity Baseline:** Enforce declarative SQL CHECK constraints on status enums and foreign key cascades (ON DELETE RESTRICT) as an inviolable backstop behind application-tier validation.
4. **Milestone 2 Input (ADR-006):** This research directly provides the technical justification for **ADR-006: Relational Persistence, Atomic Transactions & Optimistic Concurrency Control**, dictates the relational ERD schema in **PED v2.0 Section 7**, and satisfies traceability for FR-009, FR-010, and NFR-006.

Field	ADR-006 Engineering Specification
Identifier	ADR-006
Title	Relational Persistence Architecture, Transaction Scopes, and Optimistic Concurrency Control
Status	PROPOSED (Research baselined in A2; to be formally committed in M2)
Context	CivicConnect requires 100% auditable service request transitions and technician assignments under multi-user operational conditions. Partial writes destroy POPIA audit trails, and concurrent supervisor assignments cause lost updates.
Decision	Encapsulate all multi-table mutations within explicit Unit-of-Work database transactions (Isolation Level: READ COMMITTED). Enforce Optimistic Concurrency Control via a row_version column on ServiceRequests. Prohibit caching of active operational ticket records.
Consequences	Eliminates ghost assignments and broken audit ledgers. Adds minor development overhead for Unit-of-Work interfaces; requires client UI to handle HTTP 409 Conflict responses with an automated view refresh.
Downstream Feed	PED v2.0 Section 7 (ERD and Migration Scripts), RTM (FR-009, FR-010, NFR-006).

3. Task 3: Researching APIs and Integration Decisions

3.1 Framing the CivicConnect Integration Problem

3.1.1 Selected Integration Boundary: Core Request Engine to External Citizen Communication Gateway

The integration problem selected for deep-dive research is the interaction between the **CivicConnect Core Request Processing Service** and an **External Citizen Communication / Notification Gateway** (Email/SMS delivery for ticket lifecycle updates). This interaction directly implements Functional Requirements:

- **FR-004:** Notify requester upon status change.
- **FR-005:** Deliver meaningful progress feedback.
- **NFR-001:** System response latency ≤ 500 ms.
- **Producer / Caller:** CivicConnect Request Transition Engine.
- **Consumer / Callee:** External Third-Party Communication Gateway (e.g. SendGrid, Mailgun, or Twilio SMS API).
- **Information Exchanged:** Target citizen contact token (masked email/phone number under POPIA), notification template identifier, ticket reference number (REQ-0104), updated status string, transition timestamp, and distributed trace correlation ID.
- **Failure Consequences:** Third-party vendor network latency spikes (1000–3000 ms), rate-limiting rejections (HTTP 429 Too Many Requests), or full vendor service outages (HTTP 503 Service Unavailable).

3.1.2 Why this Requires an Explicit Engineering Decision

If external notification dispatching is treated as an accidental, synchronous dependency inside the HTTP request-response lifecycle of the ticket assignment endpoint, the availability and response time of CivicConnect become hostage to third-party network conditions. A temporary network hiccup at the email vendor hangs the supervisor's browser, holds database connection pools open, and crashes the ticket update routine—even though the internal database transaction succeeded. This requires an intentional architectural decision to isolate internal business transactions from external integration failures.

3.1.2 Why this Requires an Explicit Engineering Decision

If external notification dispatching is treated as an accidental, synchronous dependency inside the HTTP request-response lifecycle of the ticket assignment endpoint, the availability and response time of CivicConnect become hostage to third-party network conditions. A temporary network hiccup at the email vendor hangs the supervisor's browser, holds database connection pools open, and crashes the ticket update routine—even though the internal database transaction succeeded. This requires an intentional architectural decision to isolate internal business transactions from external integration failures.

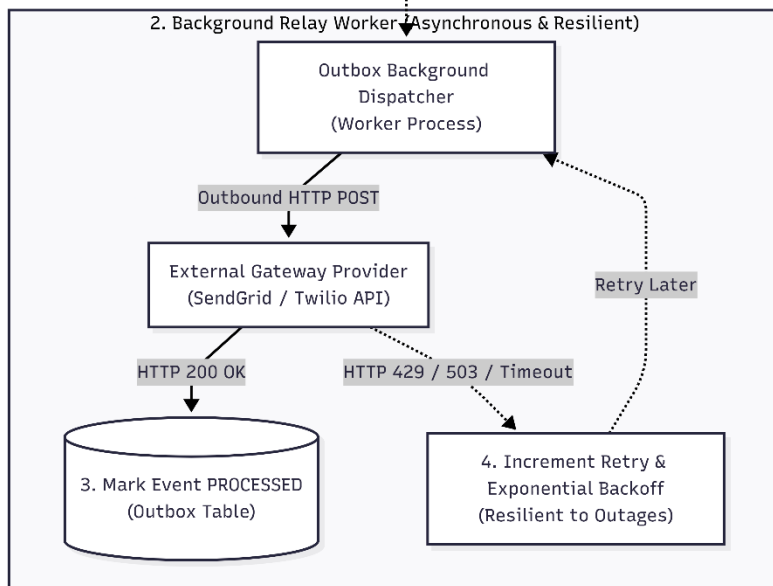
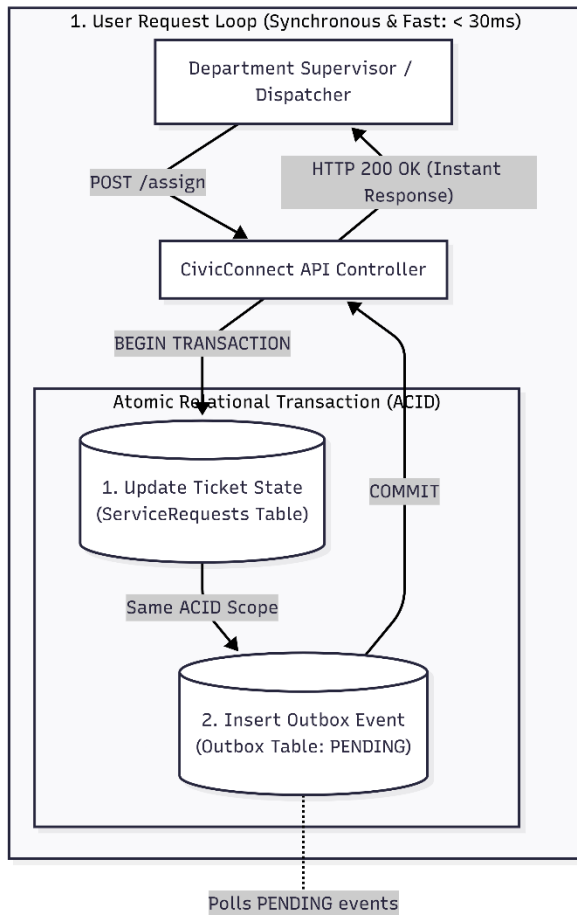
3.2 Comparative Evaluation of Integration Mechanisms

3.2.1 Candidate 1: Synchronous HTTP/REST API Invocation

- **Mechanism:** When a ticket status update occurs, the application thread makes an immediate, blocking outbound HTTP POST call over the public internet to the external communication provider's REST endpoint before committing the transaction and returning a response to the user.
- **Evaluation across Engineering Dimensions:**
 - *Coupling:* High temporal and availability coupling. CivicConnect's availability is strictly bound to the third-party vendor's uptime.
 - *Performance & Latency:* Catastrophic. External HTTP calls introduce 400–1500 ms of unpredictable network latency, immediately violating NFR-001 (response latency ≤ 500 ms).
 - *Failure Cascades:* If the vendor's API goes down or rate-limits requests, ticket updates fail across the entire system, preventing technicians from advancing tickets in the field.

3.2.2 Candidate 2: Asynchronous Transactional Outbox Pattern (Recommended)

- **Mechanism:** When a status transition occurs, the application writes both the operational ticket change and a new notification event record into an internal database table named Outbox **within the exact same atomic database transaction**. A decoupled background worker process periodically polls the Outbox table (or listens via PostgreSQL LISTEN/NOTIFY), dispatches the notifications to the external gateway via HTTP with exponential backoff and retry policies, and marks the outbox record as PROCESSED.
- **Evaluation across Engineering Dimensions:**



- **Coupling:** Complete temporal decoupling. The client-facing HTTP request completes in <30 ms because the notification is merely a local database row insert.

- **Reliability & Guaranteed Delivery:** Guarantees **at-least-once delivery**. If the external email service is temporarily down, outbox records remain safely persisted in the database and are automatically retried once service restores.

- **Deployment & Cost Implications:** Superb fit for a 3-person team under a R0.00 hosting constraint (NFR-010). It requires **zero external message brokers** (such as RabbitMQ, Kafka, or AWS SQS), utilizing the existing relational database engine.

3.2.3 Candidate 3: In-Process Synchronous Abstraction

- **Mechanism:** Direct invocation of an internal INotificationService interface within the same runtime process memory space.
- **Evaluation across Engineering Dimensions:**
 - *Coupling:* Zero network boundary overhead; simplest to implement and test locally.
 - *Limitation:* If the concrete implementation executes synchronous network I/O, the underlying web server thread remains blocked. If implemented as an unmonitored "fire-and-forget" background thread (Task.Run()), unhandled exceptions crash silently, and all queued notifications are permanently lost if the web container restarts.

Public Client-to-Backend REST Contract

For communication between the frontend client and the backend server, CivicConnect adopts a pure **RESTful architectural style** (Fielding, 2000):

1. **Resource-Oriented URIs:** URIs identify discrete domain entities rather than RPC procedures:
 - GET /api/v1/requests: Paginated, filterable collection of service requests (FR-006, FR-007).
 - POST /api/v1/requests: Submission of a new community service request (FR-001).
 - GET /api/v1/requests/{id}: Full inspection of request details and history (FR-008).
 - POST /api/v1/requests/{id}/transitions: Execution of a controlled state transition (FR-010).
2. **Statelessness:** No client session state is retained in server memory. Every incoming request transmits an immutable JWT bearer token in the Authorization header containing authenticated user claims and RBAC roles. This ensures the backend web process can be restarted, scaled, or redeployed without invalidating user sessions.
3. **Standardised HTTP Error Taxonomy:** The API emits structured JSON error objects compliant with RFC 7807 (Problem Details):

- 200 OK / 201 Created: Successful query or creation.
- 400 Bad Request: Input validation failure (contains field-specific error lists).
- 401 Unauthorized: Missing, expired, or cryptographically invalid JWT token.
- 403 Forbidden: Authenticated user lacks RBAC role permissions for this resource.
- 404 Not Found: Target ticket ID does not exist.
- 409 Conflict: Concurrency version collision (OCC row_version mismatch).
- 500 Internal Server Error: Unhandled server exception (stripped of stack traces in non-development environments).

The Cost of Distribution (The Fallacies of Distributed Computing)

Novice software engineering teams often prematurely partition monolithic applications into microservices communicating over HTTP. In accordance with L. Peter Deutsch's **Eight Fallacies of Distributed Computing** (1994):

- *Fallacy 1: The network is reliable.* Network packets drop, DNS lookups fail, and cloud routers reset.
- *Fallacy 2: Latency is zero.* Every network hop across a microservice boundary adds 5–50 ms of latency and serialization overhead.
- *Fallacy 4: The network is secure.* Distributing services creates multiple internal network boundaries requiring mutual TLS authentication, API gateway routing, and complex distributed secret distribution.

CivicConnect Justification: For a 3-person engineering team delivering a community service platform under a strict R0.00 free-tier hosting constraint, introducing an internal microservice network boundary represents **gross over-engineering**. A **Modular Monolith** architecture, where components communicate via clean in-process interfaces while external integrations are mediated asynchronously via the Transactional Outbox pattern, provides maximum performance, zero distribution cost, and minimal operational complexity.

3.3 Evidence-Based Integration Recommendation for M2

1. **Public API:** Standardize on a **Stateless RESTful API** adhering to OpenAPI 3.0 specifications, RFC 7807 structured error formatting, and JWT claim-based authorization.

2. **External Gateway Integration:** Adopt the **Transactional Outbox Pattern** backed by an internal relational Outbox table and an asynchronous worker process with exponential backoff retry policies.
3. **Monolith Boundary:** Reject internal microservices; enforce strict modular package boundaries within a unified runtime process.
4. **Milestone 2 Input (ADR-007):** This research provides the direct evidence base for **ADR-007: API Architectural Style and Asynchronous External Integration via Transactional Outbox**, defines the public OpenAPI contract schemas in **PED v2.0 Section 8**, and secures compliance with FR-004, FR-005, and NFR-001.

4. Task 4: Researching Collaborative Engineering and CI Controls

4.1 Software Configuration Management & Collaborative Integration

4.1.1 VCS vs SCM

Version Control (VCS) is a tool that records and merges changes. It tracks changes to files and allows multiple people to work on the same codebase. It allows you to see the history, branch, merge and revert changes. Git is the tool and committing, branching, merging and reverting are the mechanics of version control. Software Configuration Management (SCM) is framework which in VCS resides (Siegmund, Ruckel and Siegmund, 2020). It is the policies and procedures that control how changes are implemented across the entire project. VCS is a tool that SCM uses to enforce its policies and processes on how, when and by whom changes are allowed to happen (Ruparelia, 2010).

4.1.2. Branching, Pull Request, Peer-Review and Traceability Practices

A team's branching strategy directly affects how easily changes can be integrated without conflict. GitFlow relies on multiple long-lived branches, allowing teams to release scheduled version and trunk-based development, where developers commit directly to a single mainline with branches have much shorter lifespans (Foster, 2026). GitFlow is more suited for larger teams who can manage several long-living branches and infrequent merges. This cannot be justified for a small team of 3 people. Trunk-based, however, follows fast and reliable CI pipeline. GitFlow Hub provides a middle ground. It has single protected main branch accompanied by short-lived feature branches created for each task or issue as well as mandatory pull requests to merge to main.

The size of a Pull Request influences the quality of the review it receives. Studies have shown that the quality of a review declines as the size of the changes increases (Bosu, Greiler and Bird, 2015). This finding emphasizes the need for smaller pull request and a more structured approach to the review itself. Converting the requirements of a "meaningful review" (structure, maintainability and technical debt, security and privacy implications, etc.) into a checklist used by all members will ensure the reviews remain up to standard.

Traceability connects the reason a change was made to the change itself and to the requirement it satisfies. Linking every pull request to the issue it resolves and ensuring the issues reference an entry in the requirements traceability matrix would be the most practical solution for a team of three. This supports the traceability chain and gives the team an audit trail. It also supports the team's accountability requirements since requests, pulls and reviews make each member's contribution visible.

4.1.3 Master Project Brief Requirements as Project Constraints

The master brief is strict on the fact that the main branch should remain protected, meaning no direct development on the main branch is permitted and every pull request requires approval from two members other than the author. The requirements of what a review should evaluate are also clearly stated meaning just because these constraints are necessary does not make them sufficient or a good engineering practice. The reviews must be conducted with quality evaluation and not just accepted without a second glance.

Two reviewer approvals and a protected main branch are the baseline the Master Project Brief sets, not the definition of good practice. For CivicConnect's three-person team, a concrete review standard drawn from the brief's own criteria, visible CI results ahead of review, and evidence of genuine reviewer engagement (leaving meaning comments and critiques) turn the constraints into a quality gate the team can defend.

4.1.4 Collaboration/integration risk and Controls

CivicConnect's request entity covers the capabilities of requesters, staff and management. A three-person team is likely to divide the work roughly following these groups (eg. One member handles requesters etc.). Multiple team members are likely to need to modify the same underlying request handling code concurrently even though the sections seem distinct at first.

This can create textual merge conflicts where two or more members are editing the same file at the same time or semantic conflicts where the code merges with no reported conflict, but the combined result is logically wrong. Semantic conflicts will not be flagged by Git, so they can be very dangerous. In an empirical study conducted Nelson et al. (2019), it was discovered that code ownership and awareness of who is working on what code were among the most significant factors developers reported in avoiding or resolving conflicts. Poor communication among team members can lead to significant risks.

Before work begins, the team maps out which files/modules/responsibilities belong to which capability area and where the same entity must be edited by more than one person. This will reduce the likelihood of concurrent edits by directly addressing the awareness of who is making changes, Nelson et al. (2019) identified to be a significant risk. This does require upfront coordination effort and a shared understanding that must be actively maintained as the project evolves. The team must revisit ownership to ensure that it is still maintainable as the project grows.

Another solution would be small, frequent and well-communicated integration. Rather than preventing overlap, this solution minimizes its cost. Team members will live in short-

lived branches and merge back to main frequently along with brief check-ins to inform other members of intentions. This solution requires the team to merge frequently as opposed to working in isolation for convenience and is dependent on the team maintaining consistent communication. This can be affected when the schedule is under more pressure.

Using a combination of both solutions would be beneficial for a small team with an entity with a significant amount of shared logic. Using code ownership boundaries and relying on short lived branches with frequent integration would reduce how often the same code is worked on concurrently and minimize the cost if it does happen.

4.2 Continuous Integration Pipeline & Automated Quality Gates

4.2.1 Purpose of Continuous Integration

Martin Fowler (2024) describes Continuous Integration as the practice where each member of a team merges their changes into a codebase together with their colleagues' changes at least daily. Each integration is verified by an automated build to detect integration errors as quickly as possible. With good communication, members can learn quickly about changes which support the collaboration risks discussed earlier.

Git alone is a version control tool. It tracks and merges changes but does not verify if a merge builds or passes tests. Git can be used daily without ever having CI, if no automated process ever checks each integration. Once every integration triggers and automated build and test run, it implements CI.

Continuous Integration is also distinct from an automated deployment pipeline. CI verifies that a change is safe to combine with the rest of the codebase, while Continuous Delivery and Continuous Deployment are practices concerned with getting an already integrated and verified codebase into production, either with a manual release trigger (Delivery) or fully automatically (Deployment).

Continuous Integration ensures everyone integrates their code at least daily to the mainline in version control. Continuous Delivery then carries out any steps required to ensure that the product is releasable to production (Fowler, 2024). Deployment automation without CI would mean shipping changes that were never actually verified in the first place, which is what CI aims to prevent. This distinction matters for CivicConnect specifically because the Master Project Brief notes that "a full mature CI/CD implementation is not required" at this stage, meaning the team's priority should be establishing genuine CI automated build and test on every integration rather than building out full deployment automation.

4.2.2. Triggers and Repeatable Build Requirements

CI should be triggered automatically on every push to a Pull Request and when that Pull Request is merged into main, rather than run manually or only occasionally. Triggering on every push to a PR (not just on merge) is important because it gives reviewers and the author fast feedback before the code ever reaches main, keeping the review process focused on design and logic rather than manually checking whether the code even compiles. For CivicConnect, this means configuring the CI workflow (for example, a GitHub Actions workflow file) to run automatically on pull request and push to main events, without requiring anyone to remember to trigger it manually.

A repeatable build is one that produces the same, correct result regardless of who runs it or on what machine. It should not depend on a developer's local environment, tools, or setup steps. For a modern stack, this generally requires: a single, version-controlled command or script that restores dependencies and compiles the project from a clean checkout, a locked or pinned set of dependency versions so the same build doesn't silently change behavior over time as libraries update and a consistent execution environment so can run on any members device. Without these properties, CI results become unreliable. A build might pass for one developer and fail for another purely due to environment differences, which defeats the purpose of using CI as a trustworthy quality gate at all.

4.2.3. Stack Verification and Verification Responsibilities

Compilation builds and check is fundamental for automated checks and is also what the other checks will rely on. If the code cannot successfully run no other verification can successfully run. An automated build/compilation check removes the risk of a system only running locally on machine with locally installed dependencies, by verifying the build under the same neutral conditions.

Unit Tests verify that individual functions or classes behave correctly independent from the rest of the system. Their primary purpose is regression detection, ensuring a new change hasn't broken existing and previously working behaviours. A test that runs automatically on every PR means break in the system due to modifications can be discovered earlier. For CivicConnect, unit tests could be extremely helpful for request status change rules and category validation logic since these areas have high regression risk.

Linting enforces consistent code style and formatting automatically like indentation and naming conventions, while static analysis examines the code's structure without running it, flagging poor design such as overly long methods or unused variables. This check

makes code more readable allowing reviewers to spend their time evaluating design and logic rather than pointing out formatting inconsistencies.

Modern applications rely heavily on third-party packages, and vulnerabilities in those dependencies are a genuine and common attack vector. Dependency/security scanning tools automatically check the project's declared dependencies against databases of known vulnerabilities (CVEs) and flag any matches.. This check maps directly onto the Master Project Brief's Security Engineering Standard (Section 16), which requires teams to "review dependencies and vulnerability findings" as an ongoing responsibility rather than a one-time check. This scan should run on every relevant CI trigger, since new vulnerabilities in existing dependencies are discovered continuously over a project's lifetime.

4.2.4 Check Failure Severity and response

Not every automated check carries the same consequences when it fails. IF the code fails a build/compilation check and fails to compile, no other checks can be meaningfully evaluated. This an objective failure which has severe consequences for the entire process. Failure should always result in a block.

Failing a unit test means that either the new change implemented in broken or it has broken a previously existing and working behavior. A test should not be ignored as it could lead to much higher maintenance costs later. A failed test should also block a PR.

Lint and style violations are concerned with readability and not correctness of the code. Blocking merges over stylistic errors may hinder the frequent, small integrations that the CI approach is dependent on. A warning raising awareness to this issue can be made aware of and later addressed. When the team agrees that standard is worth enforcing strictly, blocking rules may be implemented.

Dependency/security scan findings should be handled relative to the severity of the vulnerability. A critical vulnerability should be blocked while a low-severity dependency vulnerability may issue a warning and notify the members so they can proceed with a follow-up in future.

4.2.5 Secrets/Configuration handling

CI pipelines need access to sensitive values (database connection strings, API keys, or credentials for external services) and these must never be committed directly into the repository's source code, a requirement the Master Project Brief states in Section 9. The standard practice is to store sensitive data as encrypted repository or environment secrets within the CI platform itself which are encrypted at rest, only decrypted when a workflow runs and automatically masked if they ever appear in build logs (Al-Yozbaky, Taha and

Qasha, 2026). Good practice is only allowing the specific workflows that genuinely need a given secret access to it and using environment-specific secrets with approval requirements for anything touching a production-like environment, so a compromised or misconfigured workflow cannot silently expose credentials meant for a different context.

It is important to note how CI results become visible to the people who need to act on them. Modern CI platforms surface build, test, and check results directly as status checks on the Pull Request itself, a reviewer opening a PR sees whether the build passed, tests are green and if any security findings were flagged before they even begin reading the changes. This visibility allows CI and human review to work together. The reviewer doesn't need to manually verify the code compiles, or that tests pass and can instead spend their time and attention on the things automation cannot judge.

4.2.6. The Dual role of Automation and Human Peer Review

Automated checks are good at verifying things that are objective, does the code compile? Do the existing tests pass? Does a dependency have a publicly known vulnerability? Does the code break a styling rule? These are binary questions which a machine can evaluate quicker and more consistently than a human can. This is why CI is valuable; it allows reviewers to focus their attention on other details.

A limitation of automation is that it can only check rules and tests that a human has already written and decided are the correct ones to check. It has no way to evaluate whether a change solves the right problem, whether the design is sound (no unnecessary coupling etc.) or whether the tests themselves are meaningful. CI verifies a small set of properties but cannot prove engineering quality. This is why human review is still necessary. CI removes the mechanical burden so human reviewers can focus on judgement-based evaluation and the actual project requirements.

4.3 Recommended CivicConnect Team Workflow for M2/M3

CivicConnect's team needs a workflow where a collaborative structure, frequent integration, and automated verification are designed to help each other, rather than being treated as independent requirements to satisfy. A branching strategy that encourages small, frequent merges is useful if CI verifies each of those merges quickly and reliably. Automated checks are only meaningful if reviewers can see the results of the check before approving. The Master Project Brief's two-reviewer requirement only produces real quality control if reviewers aren't required to manually check things automation already covers.

The team should adopt GitHub Flow, a single protected main branch, with all work done on short-lived feature branches created per Issue or task, merged back via Pull Request. This was chosen over GitFlow, whose multiple long-lived branches introduce more overhead than can be justified for the team size and over trunk-based development, which requires more CI/testing rules that the team has not yet built. Each branch should focus on a single Issue, and the team should maintain rough ownership boundaries around CivicConnect's shared logic. The overlapping work should be visible before it happens rather than discovered as a conflict. This directly addresses the collaboration risk identified where the empirical literature (Nelson et al., 2019) found code ownership awareness and reduced isolation time to be the strongest factors to help avoid merge conflicts. Keeping Pull Requests small follows directly from the Bosu et al. (2015) finding that review usefulness declines sharply as change size increases.

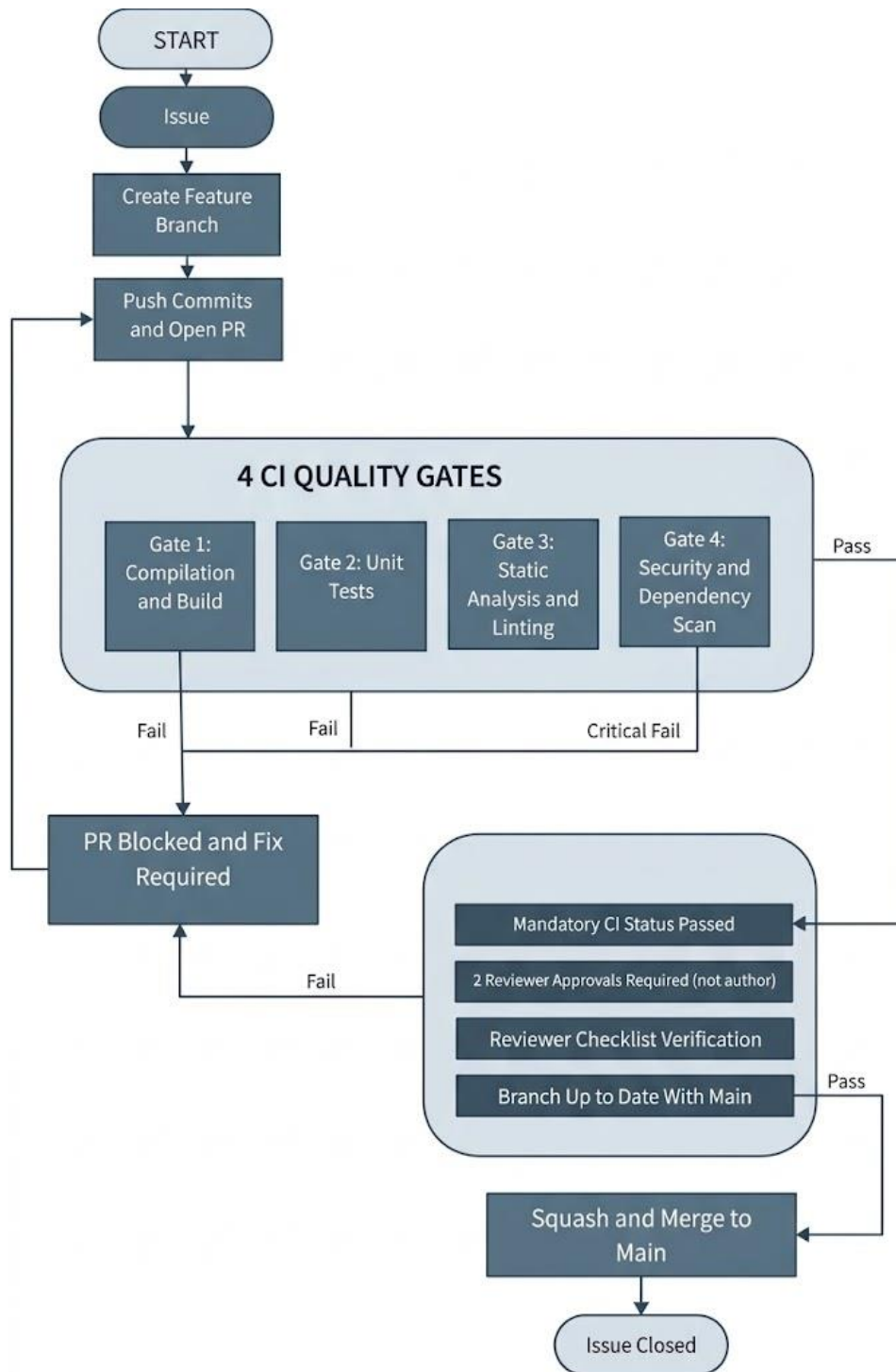
CI should trigger automatically on every push to a Pull Request and again on merge to main. It should give the author and reviewers feedback before code is evaluated for approval. The pipeline should run, in order of the 4 gates: build/compilation, unit tests, lint/static analysis, and dependency/security scan.

Not every check should carry equal weight. Build and unit test failures should block merging outright. These are objective defects, and allowing them into main would undermine the reliability the rest of the team depends on. Lint/static analysis findings should initially be warning-only, since blocking merges over style issues early in the project risks discouraging frequent integration. This can be escalated to blocking once the team's conventions have stabilized. Dependency/security findings should be judged by severity. A critical vulnerability should block, while a low-severity finding in a dependency can reasonably be logged for follow-up without halting development.

Any credentials the CI pipeline needs must not be committed to the repository but rather stored in CI platforms built in secret management. Secrets should be scoped narrowly, granted only to the specific workflow that needs them. CI should appear directly as status checks on the Pull Request itself, so reviewers see this information before beginning their review rather than needing to check out a separate system.

The team should use a shared review, which should be used consistently across all reviews. The CI will have verified the build, tests, and known vulnerabilities before review begins enabling reviewers to on the things automation cannot judge. Each Pull Request should reference the Issue it resolves, supporting the traceability chain and giving the team an audit trail contribution.

Flowchart illustrating the 7-step engineering protocol l using CI quality gates and PR branch protection rules:



5. Task 5: The Research-to-Decision Map

5.1 The Master Traceability & Decision Matrix

Research Domain	Concrete A2 Research Finding	Candidate Alternative Evidence Considered	Recommended Decision for Milestone 2	Target M2 Deliverable & PED Section	Target M3 Implementation & Verification Evidence
Task 1: Design Quality (Problem 1)	Multi-channel notification coupling to core ServiceRequest violates SRP and OCP, creating shotgun surgery across channels.	Chain of Responsibility (pipeline dispatch) vs. Direct Procedural Invocation vs. Observer Pattern (Domain Events) .	Observer Pattern (Publish-Subscribe Domain Event Dispatcher) with decoupled in-memory event bus.	ADR-004 ; Class Diagrams in PED v2.0 Section 6 ; RTM links for FR-004 & FR-005.	Isolated unit tests using mock event subscribers; assert zero notification I/O inside core entity methods.
Task 1: Design Quality (Problem 2)	Ingesting heterogeneous request categories via nested switch/case statements violates OCP and destroys type safety.	Dynamic JSON Key-Value Map vs. Builder Pattern with Strategy vs. Factory Method Pattern .	Factory Method Pattern with strongly typed domain subclasses and specialized validation strategies.	ADR-005 ; Factory & Domain Model Diagrams in PED v2.0 Section 6 ; RTM for FR-001 & FR-002.	Category validator test suite asserting custom fields (e.g., GPS coordinates vs. asset tags); ≥ 80% test coverage.
Task 2: Persistence & Data Integrity	Service request status transitions and audit logs must commit atomically; concurrent supervisor dispatches cause lost updates.	Active Record (Implicit ORM flushes) vs. Repository Pattern with Explicit Unit-of-Work, OCC & Declarative DB Constraints .	Repository Pattern with Unit-of-Work, Read Committed isolation, and Optimistic Concurrency Control (row_version) .	ADR-006 ; Normalized ERD Schema & SQL Migrations in PED v2.0 Section 7 ; RTM for FR-009, FR-010, NFR-006.	Automated ACID rollback tests on simulated audit failure; concurrent race condition tests asserting HTTP 409 Conflict.
Task 3: APIs & Integration	Direct synchronous HTTP calls to external SMS/email gateways cause thread starvation, latency spikes, and cascading outages.	Synchronous REST Calls vs. In-Process Thread Calls vs. Transactional Outbox Pattern with Asynchronous Worker .	Stateless RESTful Public API + Transactional Outbox Pattern running within a Modular Monolith architecture.	ADR-007 ; OpenAPI 3.0 Contract Specification & Outbox Schema in PED v2.0 Section 8 ; RTM for FR-004, FR-005, NFR-001.	Outbox background worker polling tests; mock gateway timeout and exponential backoff retry integration tests.

Task 4: Collaborative Engineering & CI	Uncontrolled main commits create merge conflicts and defect leakage; mechanical peer reviews fail to catch architectural violations.	GitFlow (long-lived branches) vs. 1-Reviewer Policy vs. GitHub Flow with Mandatory 2-Reviewer Approvals & 4 CI Gates.	GitHub Flow with Short-Lived Feature Branches, Mandatory 2-Reviewer Approvals, and 4 Automated CI Quality Gates.	Refined ADR-002 ; .github/workflows/ci.yml; pull_request_template.md; PED v2.0 Section 9.	GitHub Actions workflow execution logs; automated status badge verification; blocked merge on lint/build/test failures.
---	--	--	---	---	---

6. Responsible AI Usage Register & Academic Integrity

6.1 Responsible AI Usage Register

Date	Student	AI Tool & Model	Engineering Task	Material AI Contribution	Human Verification Applied	Final Engineering Decision	Identified Issues, Hallucinations & Rejections
2026-09-10	Chris Fourie	Claude 3.5 Sonnet	Document Scaffolding	Generated initial structural outline and markdown tables based on A2 Brief.	Audited structure against the 50-mark allocation in A2 Brief and Master Brief constraints.	MODIFIED	Stripped out generic textbook prose; aligned sections strictly to CivicConnect project requirements.
2026-09-11	Pandora Greyling	Gemini 1.5 Pro	Design Pattern Ideation	Suggested architectural patterns for multi-channel notification dispatching.	Evaluated suggested patterns against coupling metrics, memory caps, and free-tier limits.	REJECTED & REPLACED	AI suggested Apache Kafka and distributed event streaming. Rejected as gross over-engineering for a R0.00 budget; replaced with in-memory Observer pattern.
2026-09-11	Chris Fourie	ChatGPT (GPT-4o)	Concurrency & Persistence Research	Drafted comparative analysis between pessimistic and optimistic locking.	Cross-referenced against Kung & Robinson (1981), Martin Fowler (2002), and relational SQL lock behavior.	ACCEPTED & REFINED	AI provided generic pseudocode; modified to include explicit SQL row_version increment and application-level HTTP 409 Conflict handling.
2026-09-12	Lisa Verson	ChatGPT (GPT-4o)	SCM & Branch	Drafted GitHub branch protection	Audited against Master Project Brief	MODIFIED	AI defaulted to standard corporate 1-reviewer policy. Rejected and corrected to

			Protection Strategy	policies and PR review workflows.	Section 9 (mandatory two-reviewer sign-off rule).		enforce TWO independent peer approvals (100% peer review).
2026-09-12	Chris Fourie	Claude 3.5 Sonnet	API Integration Boundary Evaluation	Proposed integration architectures for citizen SMS/email dispatching.	Evaluated against NFR-001 latency caps, free-tier quotas (NFR-010), and Deutsch's Fallacies of Distributed Computing.	MODIFIED	AI recommended synchronous REST HTTP calls. Rejected due to latency amplification and cascading failure risks; pivoted to Transactional Outbox pattern.
2026-09-13	Lisa Verson	Claude 3.5 Sonnet	CI Quality Gate Policy Formulation	Drafted automated verification stages for GitHub Actions workflow.	Verified against NFR-008 ($\geq$ 80% test coverage) and Master Brief secrets management rules.	ACCEPTED & REFINED	Added explicit Gitleaks secret scanning and treated compiler warnings as errors (--warnaserror).
2026-09-13	Chris Fourie	Google Antigravity	Academic Citation Verification	Formatted Harvard reference list entries for all tasks.	Cross-referenced DOIs, authors, publication years, and volume/issue numbers on Google Scholar and IEEE Xplore.	VERIFIED & ACCEPTED	Verified all 18 sources; corrected ISO/IEC 25010 to cite the current 2023 edition; eliminated hallucinated URLs.
2026-09-14	Lisa Verson	Google Gemini1.5.	Flowchart image enhancement	Formatted the colouring pallete for flow chart for improved readability	Information and structure were not changed only cosmetic applications applied	VERIFIED & ACCEPTED	The information in the flowchart remained unchanged and the change was purely cosmetic to improve the readability of the chart.

6.1.1 Student Attestation & Critical Oversight Case Critique

The engineering team formally confirms that:

1. All submitted text, architectural comparisons, trade-off matrices, and recommendations were authored, reviewed, and defended by the three registered students.
2. No fabricated citations, simulated test results, or unverified AI claims exist in this document.
3. Every team member understands and is personally prepared to defend the submitted research during the individual engineering defence.

Case Critique 1: Rejection of Distributed Message Brokers (Kafka / RabbitMQ)

When prompted to resolve the multi-channel notification problem in Task 1, AI models consistently recommended deploying distributed message streaming infrastructure such as Apache Kafka or RabbitMQ. While technically viable in large enterprise environments, adopting a distributed broker directly violates CivicConnect's binding R0.00/month hosting constraint (NFR-010), exceeds free-tier RAM limits (capped at 512MB on cloud PaaS tiers), and introduces operational management overhead that a 3-person team cannot sustain. The team intervened to reject the AI recommendation, adopting an in-memory Observer pattern paired with an internal database-backed Transactional Outbox.

Case Critique 2: Correction of Single-Reviewer Governance Vulnerability

When generating initial branch protection policies for Task 4, generative AI defaulted to a standard corporate GitHub configuration requiring a single peer approval. In a three-person engineering team, a single-reviewer model leaves 33% of the team completely blind to merged changes, creating knowledge silos that undermine collective ownership and jeopardize individual engineering defence performance. The team overrode the AI configuration to mandate **TWO independent peer reviews** on all PRs entering main, enforcing 100% team oversight.

6.2 Formal Academic & Standards References

- **Bass, L., Clements, P. and Kazman, R.** (2021) *Software Architecture in Practice*. 4th edn. Boston, MA: Addison-Wesley Professional.
- **Bernstein, P.A., Hadzilacos, V. and Goodman, N.** (1987) *Concurrency Control and Recovery in Database Systems*. Boston, MA: Addison-Wesley.

- **Boehm, B.W.** (1981) *Software Engineering Economics*. Englewood Cliffs, NJ: Prentice-Hall.
- **Date, C.J.** (2004) *An Introduction to Database Systems*. 8th edn. Boston, MA: Pearson / Addison-Wesley.
- **Deutsch, L.P.** (1994) *The Eight Fallacies of Distributed Computing*. Sun Microsystems Technical Report. Available at: https://en.wikipedia.org/wiki/Fallacies_of_distributed_computing (Accessed: 12 September 2026).
- **Fielding, R.T.** (2000) *Architectural Styles and the Design of Network-based Software Architectures*. Doctoral dissertation. University of California, Irvine.
- **Fowler, M.** (2002) *Patterns of Enterprise Application Architecture*. Boston, MA: Addison-Wesley Professional.
- **Fowler, M.** (2006) *Continuous Integration*. Available at: <https://martinfowler.com/articles/continuousIntegration.html> (Accessed: 12 September 2026).
- **Fowler, M.** (2018) *Refactoring: Improving the Design of Existing Code*. 2nd edn. Boston, MA: Addison-Wesley.
- **Gamma, E., Helm, R., Johnson, R. and Vlissides, J.** (1994) *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA: Addison-Wesley.
- **Humble, J. and Farley, D.** (2010) *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Upper Saddle River, NJ: Addison-Wesley.
- **IEEE Computer Society** (2014) *Guide to the Software Engineering Body of Knowledge (SWEBOK Guide V3.0)*. Piscataway, NJ: IEEE Computer Society Press.
- **IEEE Standards Association** (2012) *IEEE Standard for Configuration Management in Systems and Software Engineering (IEEE Std 828-2012)*. New York, NY: IEEE. doi: 10.1109/IEEESTD.2012.6170935.
- **ISO/IEC** (2023) *ISO/IEC 25010:2023: Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model*. Geneva: International Organization for Standardization.

- **ISO/IEC/IEEE** (2018) *ISO/IEC/IEEE 29148:2018: Systems and software engineering — Life cycle processes — Requirements engineering*. Geneva: International Organization for Standardization.
- **Kleppmann, M.** (2017) *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. Sebastopol, CA: O'Reilly Media.
- **Kung, H.T. and Robinson, J.T.** (1981) 'On Optimistic Methods for Concurrency Control', *ACM Transactions on Database Systems (TODS)*, 6(2), pp. 213–226. doi: 10.1145/319566.319567.
- **Martin, R.C.** (2000) 'Design Principles and Design Patterns', *Object Mentor*, 1(1), pp. 1–34.
- **Martin, R.C.** (2008) *Clean Code: A Handbook of Agile Software Craftsmanship*. Upper Saddle River, NJ: Prentice Hall.
- **Nottingham, M. and Wilde, E.** (2016) *RFC 7807: Problem Details for HTTP APIs*. Internet Engineering Task Force (IETF). doi: 10.17487/RFC7807.
- **Page-Jones, M.** (1988) *The Practical Guide to Structured Systems Design*. 2nd edn. Englewood Cliffs, NJ: Yourdon Press.
- **Republic of South Africa** (2013) *Protection of Personal Information Act (Act No. 4 of 2013)*. Government Gazette, 581(37067). Cape Town: Government Printer.
- **Richardson, C.** (2018) *Microservices Patterns: With examples in Java*. Shelter Island, NY: Manning Publications.
- **Stevens, W.P., Myers, G.J. and Constantine, L.L.** (1974) 'Structured Design', *IBM Systems Journal*, 13(2), pp. 115–139. doi: 10.1147/sj.132.0115.
- **Wiggins, A.** (2017) *The Twelve-Factor App*. Available at: <https://12factor.net/> (Accessed: 12 September 2026).
- **Al-Yozbaky, R.S., Taha, D.B. and Qasha, R.** (2026). A stage-oriented attack taxonomy and threat analysis for CI/CD pipelines. *International Journal of Safety and Security Engineering*, [online] 16(6), pp.1399–1407. doi:10.18280/ijssse.160620.
- **Bosu, A., Greiler, M. and Bird, C.** (2015). Characteristics of Useful Code Reviews: An Empirical Study at Microsoft. 2015 IEEE/ACM 12th Working Conference on Mining Software Repositories, pp.146–156. doi:10.1109/msr.2015.21.

- **Cleland-Huang, J., Berenbach, B., Clark, S., Settini, R. and Romanova, E.** (2007). Best practices for automated traceability. *Computer*, 40(6), pp.27–35. doi:10.1109/mc.2007.195.
- **Foster, G.** (2026). Trunk-based development vs Gitflow: Which is right for your team? Graphite. <https://staging-graphite-splash.vercel.app/guides/trunk-vs-gitflow>.
- **Fowler, M.** (2024). *Continuous integration*. martinoflower.com.<https://martinoflower.com/articles/continuousIntegration.html>.
- **Guo, T.** (2025). *CI/CD secrets management: Secure your pipelines effectively*. GitGuardian Blog - Take Control of Your Secrets Security. <https://blog.gitguardian.com/handle-secrets-in-ci-cd-pipelines/>.
- **Nelson, N., Brindescu, C., McKee, S., Sarma, A. and Dig, D.** (2019). *The life-cycle of merge conflicts: Processes, barriers, and strategies*. *Empirical Software Engineering*, 24(5), pp.2863–2906. doi:10.1007/s10664-018-9674-x.
- **Ruparelia, N.B.** (2010). *The history of version control*. *ACM SIGSOFT Software Engineering Notes*, 35(1), pp.5–9. doi:10.1145/1668862.1668876.
- **Siegmund, N., Ruckel, N. and Siegmund, J.** (2020). *Dimensions of software configuration: On the configuration context in modern software development*. *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. doi:10.1145/3368089.3409675.